



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Cultura i Esport



IES JUAN DE GARAY

C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN
Y FORMACIÓN PROFESIONAL



Unión Europea

Fondo Social Europeo
El FSE invierte en tu futuro

UD11

INTRODUCCIÓN A BASES DE DATOS NO RELACIONALES, MONGODB

**Bases de Datos
1º DAW/DAM**

Abelardo Martínez

Elena González

Versión:260413.1211

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚙ Interesante

ÍNDICE DE CONTENIDO

2. Introducción a MongoDB.....	4
3. CONTEXTO.....	4
4. Instalación y descarga.....	5
<i>Vídeo instalación: https://www.youtube.com/watch?v=WFRK9dTeeYY.....</i>	
5. Diferencia entre Base de Datos y Colección.....	5
6. Estructuras de datos.....	5
6.1 ¿Qué es un documento JSON?.....	5
6.2 TIPOS DE DATOS POSIBLES EN BSON.....	6
7. ISODate.....	7
7.1 Insertar una fecha específica:.....	7
7.2 Consultar documentos después de una fecha:.....	7
8. Operaciones CRUD en MongoDB.....	8
8.1 Crear una base de datos y una colección.....	8
8.2 Insertar documentos.....	8
8.3 Seleccionar datos.....	8
8.3.1. Orden (sort).....	9
8.3.2. Límite (limit).....	10
8.3.3. Formato legible (pretty).....	10
8.3.4. Para acceder a un elemento de un array.....	10
8.4 Resumen de los filtros más utilizados y ejemplos.....	10
8.4.1. Operadores de Comparación.....	10
8.4.2. Operadores Lógicos.....	11
8.4.3. Operadores de Elemento.....	11
8.4.4. Operadores para Arrays.....	11
8.4.5. Operadores de Evaluación.....	11
8.4.6. Consultas en varios pasos, utilización de variables:.....	12
8.4.7. Ejemplo :.....	13
8.5 Actualizar documentos.....	13
8.5.1. \$set.....	14
8.5.2. \$addToSet.....	14
8.6 Reemplazar documentos.....	14
8.7 Eliminar documentos.....	15
8.8 algunas funciones muy comunes en su uso:.....	16
8.8.1. countDocuments.....	16
8.8.2. Distinct.....	16
8.8.3. \$sum.....	16
8.8.4. \$avg(Average / Promedio).....	17
9. Vídeos recomendados:.....	18
9.1 Las tecnologías NoSQL.....	18
https://www.youtube.com/watch?v=DxditMImfaQ	18
9.2 Curso MongoDB - Instalación.....	18
https://www.youtube.com/watch?v=WFRK9dTeeYY	18
9.3 Curso MongoDB - Conectarnos al servidor desde Mongo Compass.....	18
https://www.youtube.com/watch?v=u4vP56I3bY0	18
9.4 Crear Bases de datos en MongoDB Mongo Compass Mongo Shell.....	18
https://www.youtube.com/watch?v=w_i7f4gWhbg	18
9.5 MongoDB en una hora, aprende desde cero.....	18
https://www.youtube.com/watch?v=YyfdOX-Clf4	18
9.6 MongoDB Tutorial for Beginners 2024.....	18

UD011. INTRODUCCIÓN A BASES DE DATOS NO RELACIONALES, MONGODB

2. INTRODUCCIÓN A MONGODB

MongoDB es una base de datos NoSQL orientada a documentos que permite almacenar datos en un formato flexible similar a JSON, BSON (Binary JSON), . Es ampliamente utilizada en aplicaciones modernas debido a su escalabilidad horizontal, alta disponibilidad y facilidad de uso. Su modelo distribuido nativo lo hace ideal para big data y aplicaciones modernas que requieren manejo de datos no estructurados.

3. CONTEXTO

En el ámbito de las pequeñas y medianas empresas (PYMEs) en España, las bases de datos no relacionales (NoSQL) más utilizadas suelen ser:

MongoDB: Es una base de datos NoSQL orientada a documentos que almacena datos en formato BSON, similar a JSON. Destaca por su flexibilidad y escalabilidad, lo que la hace adecuada para aplicaciones móviles, catálogos de productos y sistemas de gestión de contenidos. Además, permite modificar la estructura de datos de forma sencilla y manejar grandes volúmenes de información no estructurada.

Redis: Es una base de datos en memoria que se caracteriza por su rapidez y eficiencia en el almacenamiento y recuperación de datos. Es especialmente útil para almacenamiento en caché, gestión de sesiones y sistemas de mensajería en tiempo real. Sin embargo, debido a que los datos se almacenan directamente en la memoria principal, requiere una cantidad considerable de memoria RAM.

Apache Cassandra: Esta base de datos distribuida está diseñada para manejar grandes volúmenes de datos en múltiples nodos sin un punto único de fallo. Ofrece alta disponibilidad y escalabilidad, lo que la hace adecuada para aplicaciones que requieren un rendimiento constante y tolerancia a fallos.

Hemos elegido **MongoDB** porque:

- No requiere esquemas fijos, facilitando el trabajo con datos no estructurados.
- Es escalable y eficiente en aplicaciones con grandes volúmenes de datos.
- Se integra bien con JavaScript y entornos de desarrollo modernos.
- Permite operaciones CRUD de manera intuitiva.

4. INSTALACIÓN Y DESCARGA

Para instalar MongoDB:

1. Windows:

Descarga desde <https://www.mongodb.com/try/download/community>

Vídeo instalación: <https://www.youtube.com/watch?v=WFRK9dTeeYY>

2. Linux:

```
sudo apt update
sudo apt install -y mongodb
sudo systemctl start mongodb
```

3. MacOS:

```
brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb-community
```

5. DIFERENCIA ENTRE BASE DE DATOS Y COLECCIÓN

Base de datos: Es un conjunto de colecciones. En MongoDB, una base de datos contiene múltiples colecciones y se crea automáticamente cuando se almacena el primer dato en una colección.

Colección: Es un conjunto de documentos dentro de una base de datos. Se puede considerar equivalente a una tabla en bases de datos relacionales, pero sin un esquema fijo.

6. ESTRUCTURAS DE DATOS

MongoDB almacena datos en **documentos JSON** dentro de **colecciones**.

6.1 ¿Qué es un documento JSON?

JSON (*JavaScript Object Notation*) es un formato ligero de intercambio de datos que se basa en texto legible por humanos y estructurado en pares clave-valor. En MongoDB, los documentos se almacenan en este formato, permitiendo una estructura flexible sin necesidad de esquemas rígidos.

Cada documento puede tener diferentes estructuras dentro de la misma colección, lo que brinda gran flexibilidad en el almacenamiento de datos.

Creación implícita (schema-less):

```
use nueva_db // Crea DB al primer insert
db.clientes.insertOne({
  nombre: "Ana",
  membresia: "Premium",
  pedidos: [{id: 101, total: 299.90}]
}) // Crea colección 'clientes' automáticamente[4]
```

Creación explícita:

```
db.createCollection("productos", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nombre", "precio"],
    properties: {
      nombre: { bsonType: "string" },
      precio: { bsonType: "decimal" }
    }
  }
})
```

6.2 TIPOS DE DATOS POSIBLES EN BSON

Los tipos posibles en BSON (Binary JSON), el formato utilizado por MongoDB para almacenar documentos, abarcan tanto los tipos de datos estándar de JSON como otros adicionales que permiten mayor flexibilidad y eficiencia. A continuación se resumen los principales tipos soportados por BSON:

- *String*: Cadenas de texto en UTF-8.
- *Integer*: Números enteros de 32 o 64 bits.
- *Double*: Números de punto flotante de 64 bits.
- *Boolean*: Valores verdadero o falso.
- *Date*: Fechas almacenadas como milisegundos desde la época Unix.
- *Null*: Valores nulos.
- *Object*: Documentos incrustados (objetos anidados).
- *Array*: Listas de valores de cualquier tipo.
- *ObjectId*: Identificadores únicos de 12 bytes, comúnmente usados como `_id`.

- *Binary data (BinData):* Datos binarios, como archivos o imágenes.
- *Code:* Código JavaScript.
- *Timestamp:* Marca de tiempo, útil para registrar modificaciones.

En resumen, BSON extiende los tipos de datos de JSON y añade soporte para fechas, datos binarios, identificadores únicos y otros tipos especializados, permitiendo a MongoDB almacenar y consultar datos de manera más eficiente y flexible que usando solo JSON.

7. ISODATE

ISODate se utiliza en MongoDB para almacenar y manejar fechas y horas en el formato estándar internacional ISO 8601 (año-mes-día, seguido de la hora en UTC). Este tipo de dato permite que las fechas sean interpretadas de manera consistente y precisa en cualquier sistema o aplicación que acceda a la base de datos, independientemente de la configuración regional o de zona horaria.

7.1 Insertar una fecha específica:

```
db.eventos.insertOne({ evento: "Concierto", fecha: ISODate("2025-06-01T20:00:00Z") })
```

7.2 Consultar documentos después de una fecha:

```
JavaScript
db.eventos.find({ fecha: { $gt: ISODate("2025-05-01T00:00:00Z") } })
```

La cadena **"2025-05-01T00:00:00Z"** está escrita en formato ISO 8601, un estándar internacional para representar fechas y horas de manera precisa y sin ambigüedades.

Significado desglosado:

- **2025:** año
- **05:** mes (mayo)
- **01** día (1 de mayo)
- **T:** separador entre la fecha y la hora
- **00:00:00:** hora, minutos y segundos (medianoche)
- **Z:** indica que la hora está en **UTC** (Tiempo Universal Coordinado)

8. OPERACIONES CRUD EN MONGODB

CRUD es un acrónimo que representa las cuatro operaciones fundamentales en bases de datos:

- Create (Crear) → Insertar nuevos datos en la base de datos.
- Read (Leer) → Recuperar datos almacenados.
- Update (Actualizar) → Modificar datos existentes.
- Delete (Eliminar) → Borrar datos de la base de datos.

8.1 Crear una base de datos y una colección

use miAgenda

```
db.createCollection("contactos")
```

8.2 Insertar documentos

// Inserta un único documento

```
db.contactos.insertOne({ "nombre": "Ana", "edad": 25, "ciudad": "Valencia" })
```

// Insertar múltiples documentos, cada documento puede tener unos campos distintos

```
db.libros.insertMany([  
  {titulo: "1984", autor: "Orwell", paginas: 328},  
  {titulo: "Fahrenheit 451", tags: ["ciencia ficción", "clásico"]}  
])
```

8.3 Seleccionar datos

La función principal para consultar documentos en MongoDB es **find()**.

Su sintaxis básica es:

```
db.coleccion.find(<filtro>, <proyección>, <opciones>)
```

// Por ejemplo en la consulta con proyección, filtro compuesto y opciones

```
db.peliculas.find(  
  {genero: "acción", rating: {$gte: 7}},  
  {titulo: 1, director: 1, _id: 0}
```



```
).sort({año: -1}).limit(5).pretty()
```

<filtro>: (opcional) Objeto que define los criterios de búsqueda.

```
db.peliculas.find(  
  {genero: "acción", rating: {$gte: 7}},  
  {titulo: 1, director: 1, _id: 0}  
)
```

Los resaltado en amarillo serían los *filtros* de esta consulta, en esta caso sólo muestra las películas de género acción y con un rating mayor o igual a 7,

<proyección>: (opcional) Objeto que especifica qué campos incluir o excluir en los resultados.

```
db.peliculas.find(  
  {genero: "acción", rating: {$gte: 7}},  
  {titulo: 1, director: 1, _id: 0},  
)
```

Los resaltado en amarillo sería la *proyección*, que especifica qué campos deben incluirse (con 1) o excluirse (con 0) en el resultado, es decir se mostraría el título y director pero no el _id.

EJEMPLO:

```
db.amigos.find({}, {nombre: 1, email: 1, _id: 0})//En este caso, no hay  
filtro y se muestran solo nombre y email, y se omite el campo_id
```

<opciones>: (opcional) Parámetros adicionales como orden, límite, etc

```
db.peliculas.find(  
  {genero: "acción", rating: {$gte: 7}},  
  {titulo: 1, director: 1, _id: 0}  
)
```

Los resaltado en amarillo sería ordenado por año descendentemente, sólo devuelve 5 documentos y lo hace con un formato de lectura más fácil para consola.

8.3.1.Orden (sort)

El método `sort()` en MongoDB permite ordenar los resultados de una consulta según uno o varios campos.

Para ordenar de forma ascendente, se usa 1; para descendente, -1.

```
db.amigos.find().sort({nombre: 1}) // Ordena por nombre de A a Z
db.amigos.find().sort({edad: -1})  // Ordena por edad de mayor a menor
```

8.3.2. Límite (limit)

El método `limit()` restringe la cantidad de documentos que devuelve una consulta.

Es útil para trabajar con grandes colecciones y mejorar el rendimiento.

```
db.amigos.find().limit(3) // Devuelve solo los primeros 3 documentos
```

8.3.3. Formato legible (pretty)

El método `pretty()` muestra los resultados de la consulta en un formato más legible y estructurado, facilitando su lectura en la consola

```
db.amigos.find().pretty()
```

8.3.4. Para acceder a un elemento de un array

Supón que tienes en la colección `estudiantes`:

```
db.estudiantes.insertMany([ { "nombre": "Ana", "notas": [10, 8, 7] },
                             { "nombre": "Luis", "notas": [9, 6, 8, 7] }
                           ] })
```

```
//Si quieres encontrar los estudiantes cuya primera nota es 10, puedes hacer:
db.estudiantes.find({ "notas.0": 10 })
```

```
//Si quieres ver los alumnos que tengan 3 o más notas guardadas.
```

```
db.estudiantes.find({ "notas.2": { $exists: true } })
```

8.4 Resumen de los filtros más utilizados y ejemplos

8.4.1. Operadores de Comparación

Operador	Descripción	Ejemplo
<code>\$eq</code>	Igual a	<code>db.usuarios.find({edad: {\$eq: 30}})</code>

Operador	Descripción	Ejemplo
\$ne	Distinto de	db.usuarios.find({nombre: {\$ne: "Ana"}})
\$gt	Mayor que	db.usuarios.find({edad: {\$gt: 25}})
\$gte	Mayor o igual que	db.usuarios.find({edad: {\$gte: 18}})
\$lt	Menor que	db.usuarios.find({edad: {\$lt: 40}})
\$lte	Menor o igual que	db.usuarios.find({edad: {\$lte: 65}})
\$in	Valor dentro de un array	db.usuarios.find({nombre: {\$in: ["Ana", "Luis"]}})
\$nin	Valor no está en un array	db.usuarios.find({edad: {\$nin: [20,30]}})

8.4.2. Operadores Lógicos

Operador	Descripción	Ejemplo
\$and	Todas las condiciones se cumplen	db.usuarios.find({\$and: [{edad: {\$gt: 18}}, {ciudad: "Madrid"}]})
\$or	Alguna condición se cumple	db.usuarios.find({\$or: [{ciudad: "Madrid"}, {ciudad: "Barcelona"}]})
\$not	No se cumple la condición	db.usuarios.find({edad: {\$not: {\$gt: 30}}})
\$nor	Ninguna condición se cumple	db.usuarios.find({\$nor: [{ciudad: "Madrid"}, {ciudad: "Barcelona"}]})

8.4.3. Operadores de Elemento

Operador	Descripción	Ejemplo
\$exists	El campo existe o no	db.usuarios.find({telefono: {\$exists: true}})
\$type	El campo es de un tipo específico	db.usuarios.find({edad: {\$type: "int"}})

8.4.4. Operadores para Arrays

Operador	Descripción	Ejemplo
\$in	Algún elemento está en el array	db.usuarios.find({intereses: {\$in: ["deporte"]}})
\$all	Todos los elementos están en el array	db.usuarios.find({intereses: {\$all: ["deporte", "música"]}})
\$size	El array tiene un tamaño específico	db.usuarios.find({intereses: {\$size: 3}})
\$elemMatch	Algún elemento cumple varias condiciones	db.usuarios.find({notas: {\$elemMatch: {\$gt: 7, \$lt: 10}}})

8.4.5. Operadores de Evaluación

Operador	Descripción	Ejemplo
\$regex	Coincidencia por expresión regular	db.usuarios.find({nombre: {\$regex: /^A/}})
\$text	Búsqueda de texto completo	db.articulos.find({\$text: {\$search: "receta"}})

Ejemplo :

El comando `db.prestamos.find({ objeto: /libro/i })` en MongoDB busca y devuelve todos los documentos de la colección `prestamos` cuyo campo `objeto` contiene la palabra "libro", sin importar si está en mayúsculas o minúsculas.

objeto: `/libro/i` utiliza una expresión regular (regex) para buscar coincidencias de texto.

- Las barras `/` encierran el patrón a buscar, en este caso "libro".
- La opción final `i` indica que la búsqueda es *insensible a mayúsculas y minúsculas* (por ejemplo, encontrará "Libro", "libro", "LIBRO", etc.).

El primer parámetro del método `find` es el filtro de búsqueda. Aquí, selecciona los documentos cuyo campo `objeto` contenga la cadena "libro" en cualquier parte del texto.

Ejemplo de funcionamiento:

Si tienes estos documentos en la colección:

```
json
{ objeto: "Libro de matemáticas", fecha: "2024-06-01" }
{ objeto: "libro de historia", fecha: "2024-07-15" }
{ objeto: "Cuaderno", fecha: "2024-08-10" }
```

La consulta devolverá los dos primeros documentos, porque ambos contienen la palabra "libro" en el campo `objeto`, sin importar si la inicial es mayúscula o minúscula.

8.4.6. Consultas en varios pasos, utilización de variables:

Podemos crear variables para almacenar resultados intermedios, vamos a verlo con un ejemplo:

```
const amigosConPrestamos = db.prestamos.distinct("prestadoA")
db.amigos.find({ nombre: { $nin: amigosConPrestamos } })
```

Explicación paso a paso

1. Obtener los nombres de amigos con préstamos

```
const amigosConPrestamos = db.prestamos.distinct("prestadoA")
```

Este comando busca en la colección `prestamos` y obtiene **todos los valores únicos** del campo `prestadoA` (que se supone contiene el nombre del amigo al que le has prestado algo).

El resultado es un array con los nombres de los amigos que tienen al menos un préstamo registrado.

2. Buscar amigos que NO tienen préstamos

```
db.amigos.find({ nombre: { $nin: amigosConPrestamos } })
```

distinct("prestadoA"): obtiene una lista de todos los amigos con préstamos registrados.

\$nin: filtra para mostrar solo los amigos cuyo nombre **no** aparece en esa lista.

8.4.7. Ejemplo :

Buscar usuarios mayores de 25 años que vivan en Madrid o Barcelona y tengan el campo "telefono":

```
db.usuarios.find({  
  edad: {$gt: 25},  
  ciudad: {$in: ["Madrid", "Barcelona"]},  
  telefono: {$exists: true}  
})
```

8.5 Actualizar documentos

*// Actualizar primer documento coincidente, con un total menor de 50, se le añade el campo descuento y si
// ya existía se le pone de nuevo valor 10*

```
db.ventas.updateOne(  
  {total: {$lt: 50}},  
  {$set: {descuento: 10}}  
)
```

*// Actualizar todos los documento coincidentes con un total menor de 50, se le añade el campo descuento y si
// ya existía se le pone de nuevo valor 10*

```
db.ventas.updateMany(  
  {total: {$lt: 50}},  
  {$set: {descuento: 10}}  
)
```

8.5.1.\$set

- **Función:** Asigna un valor a un campo, **reemplazando completamente su valor anterior**.
- **Se usa para:** Establecer o modificar un campo con cualquier tipo de valor (número, string, array, objeto...).

Ejemplo:

```
db.usuarios.updateOne(  
  { nombre: "Ana" },  
  { $set: { ciudad: "Valencia" } }  
)
```

- ♦ Si ciudad no existía, lo crea.
- ♦ Si ya existía, lo reemplaza por "Valencia".

8.5.2.\$addToSet

- **Función:** Añade un valor a un array **solo si no existe ya dentro de él** (evita duplicados).
- **Se usa para:** Insertar elementos en arrays sin repetir valores.

Ejemplo:

```
db.usuarios.updateOne(  
  { nombre: "Ana" },  
  { $addToSet: { aficiones: "lectura" } }  
)
```

- ♦ Si aficiones es un array y ya contiene "lectura", no hace nada.
- ♦ Si no contiene "lectura", lo añade.

8.6 Reemplazar documentos

// Reemplazar documento completo

```
db.perfiles.replaceOne(  
  {user_id: "U-123"},  
  {nombre: "Carlos", pais: "México", ultimo_login: new Date()}  
)
```

```
// NO EXISTE replaceMany
```

La diferencia principal entre `replaceOne` y `updateOne` en MongoDB es cómo modifican los documentos que coinciden con el filtro:

Método	¿Qué hace?	Ejemplo de uso
updateOne	Modifica uno o varios campos específicos de un documento usando operadores como <code>\$set</code> .	<code>db.clientes.updateOne({nombre: "Ana"}, {\$set: {ciudad: "Madrid"}})</code>
replaceOne	Reemplaza el documento completo (excepto el campo <code>_id</code>) por uno nuevo.	<code>db.clientes.replaceOne({nombre: "Ana"}, {nombre: "Ana", ciudad: "Madrid", edad: 30})</code>

UpdateOne permite modificar solo ciertos campos sin perder el resto de la información del documento original. Utiliza operadores como `$set`, `$inc`, etc. Si solo necesitas cambiar o agregar campos, este método es más eficiente y seguro.

ReplaceOne sustituye todo el documento original (menos el `_id`) por el nuevo documento que le pases. Todos los campos que no estén en el nuevo documento se perderán. Es útil cuando necesitas reestructurar completamente el documento

8.7 Eliminar documentos

```
// Eliminar uno
```

```
db.usuarios.deleteOne({ nombre: "Carlos" })
```

```
// Eliminar muchos usuarios inactivos desde "2024-01-01"
```

```
db.usuarios.deleteMany({  
  ultimo_acceso: {$lt: ISODate("2024-01-01")}  
})
```

8.8 algunas funciones muy comunes en su uso:

8.8.1.countDocuments

Cuenta cuántos documentos cumplen una condición en una colección.

Uso básico:

```
db.coleccion.countDocuments ({ filtro })
```

Ejemplo:

//Cuenta cuántos estudiantes tienen más de 18 años.

```
db.estudiantes.countDocuments({ edad: { $gt: 18 } })
```

8.8.2.Distinct

Devuelve un array con todos los valores únicos de un campo en una colección.

Uso básico:

```
db.coleccion.distinct("campo", { filtro })
```

Ejemplo:

//Devuelve una lista con los nombres de todos los estudiantes, sin repetir.

```
db.estudiantes.distinct("nombre")
```

8.8.3.\$sum

Suma los valores de un campo numérico usando el framework de agregación (este framework no se va a estudiar en este tema, por no alargarlo, aunque tiene numerosas utilizaciones, para funciones más avanzadas).

Uso básico:

También se usa dentro de \$group:

```
db.coleccion.aggregate([
  { $group: { _id: null, suma: { $sum: "$campo" } } }
])
```

Ejemplo:

//Sumar todos los importes de ventas:

```
db.ventas.aggregate([
  { $group: { _id: null, totalImporte: { $sum: "$importe" } } }
])
```


8.8.4.\$avg(Average / Promedio)

Calcula el promedio de los valores de un campo numérico usando el framework de agregación (este framework no se va a estudiar en este tema, por no alargarlo, aunque tiene numerosas utilizaciones, para funciones más avanzadas)..

Uso básico:

Debes usarlo dentro de una etapa \$group en una consulta de agregación:

```
db.coleccion.aggregate([
  { $group: { _id: null, promedio: { $avg: "$campo" } } }
])
```

Ejemplo:

Supón que tienes una colección ventas con un campo importe:

//Calcula el promedio de todos los importes de ventas.

```
db.ventas.aggregate([
  { $group: { _id: null, promedioImporte: { $avg: "$importe" } } }
])
```

9. VÍDEOS RECOMENDADOS:

He elegido este canal porque me ha parecido muy concreto, muy actualizado y que está muy bien explicado para el nivel en el que estamos trabajando.

Canal de Christian Gámez

9.1 Las tecnologías NoSQL

<https://www.youtube.com/watch?v=DxditMImfaQ>

9.2 Curso MongoDB - Instalación

<https://www.youtube.com/watch?v=WFRK9dTeeYY>

9.3 Curso MongoDB - Conectarnos al servidor desde Mongo Compass

<https://www.youtube.com/watch?v=u4vP56I3bY0>

9.4 Crear Bases de datos en MongoDB Mongo Compass Mongo Shell

https://www.youtube.com/watch?v=w_i7f4gWhbq

9.5 MongoDB en una hora, aprende desde cero.

Si ya te has instalado puedes ir al minuto 20, te quedarían sólo 45 minutos muy interesantes que te sirven para introducirte en MongoDB.

<https://www.youtube.com/watch?v=YyfdOX-CIf4>

9.6 MongoDB Tutorial for Beginners 2024

Duración: Curso completo con más de 50 laboratorios.

Contenido: Desde la instalación hasta operaciones CRUD, modelado de datos, uso de MongoDB Compass y conexión con frameworks como Python y Spark.

Ideal para: Usuarios que buscan una introducción detallada y progresiva a MongoDB.

Enlace: [MongoDB Tutorial for Beginners 2024](#)